IoT Fleet Monitoring System

SAP Business Technology Platform Architecture Simulation

A Full-Stack Implementation Demonstrating SAP BTP Service Architecture Using Open-Source Tools

---

Name: Dastageer Siddiqui
Roll No: 23053275
Course: OE – SAP Business Technology Platform
Semester: 6th Semester
Date: April 2026

KIIT University

School of Computer Engineering

Contents

1 Introduction **2**
- 1.1 Objective . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Scope . . . 2

2 SAP BTP Service Mapping **2**
- 2.1 SAP CAP Reuse Model (O2C Pattern) . . . 3

3 System Architecture **4**
- 3.1 High-Level Architecture . . . 4
- 3.2 Event-Driven Data Flow . . . 4
- 3.3 WebSocket Real-Time Communication . . . 4

4 Database Design (HANA Simulation) **5**

5 Authentication and Authorization (XSUAA Simulation) **5**
- 5.1 JWT Token Flow . . . 5
- 5.2 Role-Based Access Control . . . 5

6 Workflow Management (SAP Workflow Simulation) **6**
- 6.1 State Machine . . . 6
- 6.2 Workflow Triggers . . . 6

7 Alert Processing Pipeline **6**
- 7.1 Threshold Configuration . . . 6
- 7.2 Processing Flow . . . 7

8 Telemetry Simulation Engine **7**
- 8.1 Vehicle State Model . . . 7
- 8.2 Fault Injection . . . 7

9 Frontend Dashboard (SAP Fiori Simulation) **7**
- 9.1 Components . . . 8
- 9.2 Real-Time Updates . . . 8

10 REST API Design (CAP OData Simulation) **8**

11 Technology Stack **9**

12 Setup and Execution **9**
- 12.1 Prerequisites . . . 9
- 12.2 Installation . . . 9
- 12.3 Running the Application . . . 10
- 12.4 Demo Credentials . . . 10

13 Conclusion **10**

1. Introduction

1.1 Objective

The objective of this project is to design and implement a production-grade **IoT Fleet Monitoring System** that faithfully simulates the architecture and service patterns of **SAP Business Technology Platform (BTP)**. Rather than requiring paid SAP cloud subscriptions, the system uses locally-runnable, open-source equivalents for every core BTP service, demonstrating practical understanding of SAP's enterprise architecture.

1.2 Problem Statement

Fleet management companies need to monitor vehicle health metrics (speed, engine temperature, fuel level, battery voltage) in real time. When sensor readings breach configurable thresholds, the system must automatically generate alerts and create maintenance work orders that follow a structured approval workflow. This is a classic IoT-to-action pipeline that SAP BTP is designed to handle through services like HANA, Event Mesh, XSUAA, and Workflow Management.

1.3 Scope

The system covers:

1. Real-time telemetry ingestion and storage (SAP HANA simulation)
2. Event-driven processing via pub/sub channels (SAP Event Mesh simulation)
3. Threshold-based alert generation with configurable rules
4. Automated maintenance workflow with 5-step approval flow (SAP Workflow simulation)
5. Role-based access control with JWT authentication (SAP XSUAA simulation)
6. Live dashboard with WebSocket-driven updates (SAP Fiori simulation)

2. SAP BTP Service Mapping

Each component of this project maps directly to a specific SAP BTP service. The table below documents this mapping:

Table 1: SAP BTP Service to Local Simulation Mapping

SAP BTP Service	Local Simulation	Implementation Details
SAP HANA Cloud	SQLite via sql.js (WASM)	In-memory relational database with 5 normalized tables, foreign key constraints, and performance indexes
SAP Event Mesh	Node.js EventEmitter	Singleton pub/sub broker with 3 named channels: `telemetry`, `alerts`, `maintenance`
SAP XSUAA	JWT + bcrypt	Token-based authentication with 3 role collections: Admin, Operator, Technician
SAP Workflow Mgmt	State machine	5-step approval flow: Created → Approved → Assigned → In-Progress → Completed
SAP CAP (Node.js)	Express + TypeScript	Service-oriented backend with separate service classes for each domain entity
SAP Fiori / UI5	React + Vite	Dark-themed dashboard with real-time gauges, alerts panel, and workflow management UI
SAP IoT Services	Custom simulator	Telemetry generator with realistic physics-based vehicle state evolution

2.1 SAP CAP Reuse Model (O2C Pattern)

The project adapts SAP's **Order-to-Cash (O2C)** reuse model from the Cloud Application Programming Model to an IoT domain:

Table 2: O2C to IoT Domain Mapping

O2C Entity	IoT Equivalent	Rationale
SalesOrders	`devices` table	Fleet vehicles represent the primary business objects
SalesOrderItems	`telemetry` table	Individual sensor readings are line items per vehicle
Delivery Workers	Alert Service	Evaluates conditions and triggers downstream actions
Invoice Workers	Workflow Service	Creates and manages maintenance work orders

3. System Architecture

3.1 High-Level Architecture

The system follows a **three-tier architecture** with an event-driven processing pipeline:

1. **Presentation Layer** – React SPA served by Vite, communicating via REST and WebSocket
2. **Application Layer** – Express.js backend with CAP-like service classes, JWT middleware, and Event Mesh pub/sub
3. **Data Layer** – SQLite database (HANA simulation) accessed through a thin query abstraction

3.2 Event-Driven Data Flow

The core innovation is the event-driven pipeline that mirrors how SAP Event Mesh decouples microservices:

1. **Telemetry Generator** simulates 5 fleet vehicles, publishing sensor data to the `telemetry` Event Mesh channel every 3 seconds.
2. **Telemetry Service** (channel subscriber) receives sensor data, persists it to the HANA simulation, and evaluates 4 configurable threshold rules.
3. If any threshold is breached, the service publishes an alert event to the `alerts` channel.
4. **Alert Service** (channel subscriber) persists the alert record and broadcasts it to connected WebSocket clients.
5. For *critical*-severity alerts, the Alert Service publishes a maintenance trigger to the `maintenance` channel.
6. **Workflow Service** (channel subscriber) auto-creates a work order with status "Created".
7. The work order then progresses through the SAP Workflow approval states via user actions in the dashboard.

3.3 WebSocket Real-Time Communication

The WebSocket server maintains persistent connections with all dashboard clients and broadcasts the following message types:

- `telemetry` – Live sensor data for gauge updates
- `alert` – New threshold breach notifications
- `alert_updated` – Acknowledgement status changes
- `workorder` – New work order creation
- `workorder_updated` – Workflow state transitions

4. Database Design (HANA Simulation)

The SQLite database simulates SAP HANA Cloud with 5 normalized tables:

Table 3: Database Schema Overview

Table	SAP Equivalent	Key Columns
`users`	XSUAA user store	id, username, password_hash, role (Admin/Operator/Technician)
`devices`	Device registry	id, vehicle_id, name, type (Truck/Van/Car), status, GPS coordinates
`telemetry`	Time-series store	id, device_id (FK), speed, fuel_level, engine_temp, battery_voltage, odometer
`alerts`	Alert log	id, device_id (FK), telemetry_id (FK), severity, metric, threshold, actual_value
`work_orders`	Workflow instances	id, device_id (FK), alert_id (FK), assigned_to (FK), status (5 states), priority

Seven performance indexes are created on frequently queried columns (device_id, recorded_at, severity, status, assigned_to) to simulate HANA's indexing behavior.

5. Authentication and Authorization (XSUAA Simulation)

5.1 JWT Token Flow

Authentication simulates the SAP XSUAA OAuth2 flow:

1. User submits credentials to `POST /api/auth/login`
2. Server validates password hash (bcrypt) and generates a JWT token containing user ID, username, and role
3. Client stores the token in `localStorage` and includes it in the `Authorization` header for subsequent requests
4. JWT middleware validates the token on every protected API call

5.2 Role-Based Access Control

Three roles map to SAP XSUAA role collections:

Table 4: RBAC Role Definitions

Role	SAP Equivalent	Permitted Actions
Admin	FleetAdmin collection	Full access: approve work orders, assign technicians, manage simulation, inject faults
Operator	FleetOperator collection	Monitor fleet, start/stop simulation, assign technicians
Technician	FleetTechnician collection	View assigned work orders, start and complete assigned tasks

6. Workflow Management (SAP Workflow Simulation)

6.1 State Machine

The maintenance workflow implements a 5-step approval flow, simulating SAP Workflow Management Service:

`Created` → `Approved` → `Assigned` → `InProgress` → `Completed`

Each transition validates the current state and required role before allowing the update. Invalid transitions are rejected with descriptive error messages.

6.2 Workflow Triggers

- **Auto-creation:** Critical alerts automatically trigger work order creation via the Event Mesh `maintenance` channel
- **Approval:** Admin reviews and approves the maintenance request
- **Assignment:** Admin or Operator assigns a Technician from the available pool
- **Execution:** Assigned Technician starts work, updating status to InProgress
- **Completion:** Technician completes the work with resolution notes

7. Alert Processing Pipeline

7.1 Threshold Configuration

Four vehicle metrics are continuously evaluated against configurable thresholds:

Table 5: Alert Threshold Configuration

Metric	Warning Threshold	Critical Threshold	Direction
Engine Temperature	$> 95°C$	$> 105°C$	Upper bound
Speed	> 100 km/h	> 130 km/h	Upper bound
Fuel Level	$< 15\%$	$< 5\%$	Lower bound
Battery Voltage	< 11.5 V	< 10.5 V	Lower bound

7.2 Processing Flow

Each telemetry reading is evaluated as follows:

1. Store raw telemetry data in the HANA simulation
2. Evaluate all 4 metrics against their thresholds
3. Determine aggregate device status (OK, Warning, or Critical)
4. Update the device status in the database
5. Broadcast telemetry via WebSocket to connected dashboards
6. For each breach, publish an alert to the Event Mesh `alerts` channel
7. Alert Service persists the alert and broadcasts via WebSocket
8. For critical alerts, publish to the `maintenance` channel to auto-create work orders

8. Telemetry Simulation Engine

8.1 Vehicle State Model

Each simulated vehicle maintains an internal state vector:

$$\mathbf{S}_t = [\text{speed}, \text{fuel}, \text{engineTemp}, \text{lat}, \text{lon}, \text{battery}, \text{odometer}, \text{heading}]$$

The state evolves every 3 seconds using physics-based rules:

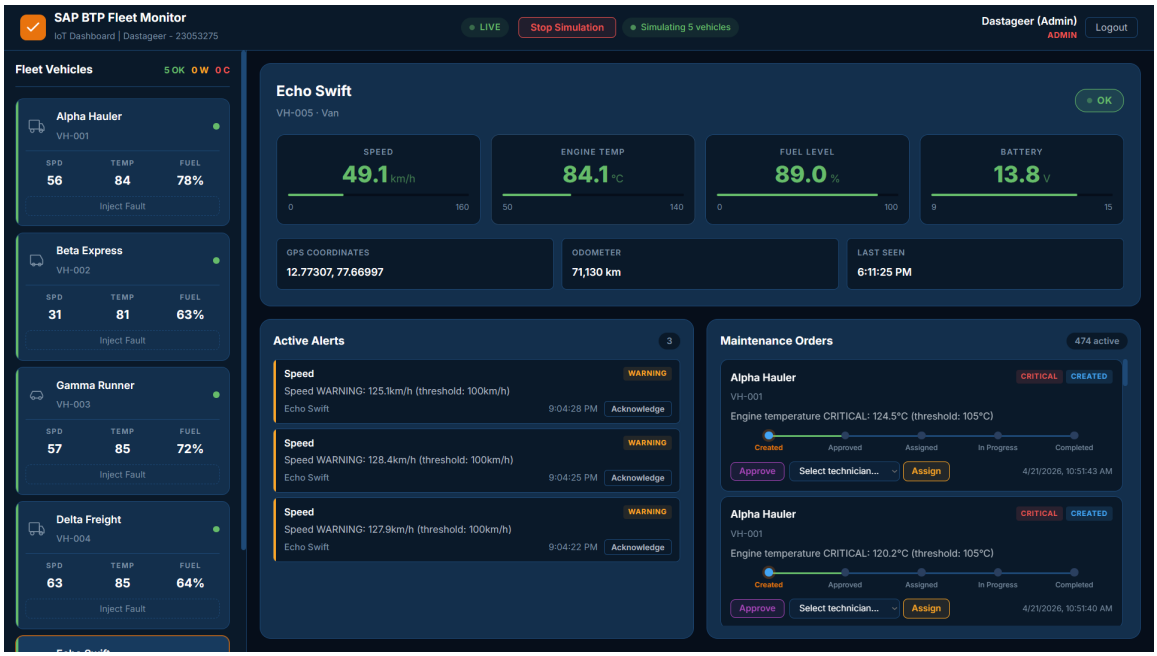
- **Speed:** Random walk with bounds $[0, 140]$ km/h, slight upward bias
- **Fuel:** Decreases proportionally to speed (consumption $\propto$ speed/60), auto-refills at zero
- **Engine Temperature:** Approaches a target based on speed ($78 + \frac{\text{speed}}{140} \times 15$) with noise. Under fault injection, temperature rises rapidly toward 135°C.
- **GPS:** Position updates based on speed, heading, and Earth geometry ($\Delta\text{lat} = \frac{d}{111} \cos\theta$)
- **Battery:** Random walk bounded to $[10.0, 14.8]$ V

8.2 Fault Injection

The "Inject Fault" feature simulates a sensor anomaly by forcing the engine temperature to rise rapidly. The fault auto-clears after 30 seconds, returning the vehicle to normal operating temperatures. This demonstrates the full alert-to-resolution pipeline.

9. Frontend Dashboard (SAP Fiori Simulation)

The frontend is a React single-page application styled with a dark theme inspired by SAP Fiori design guidelines.



9.1 Components

Table 6: Dashboard UI Components

Component	Functionality
`LoginPage`	XSUAA-style authentication with quick-login buttons for demo
`Navbar`	Navigation bar with simulation controls, connection status, and user info
`DeviceList`	Sidebar showing fleet vehicles with SVG vehicle icons and live mini-metrics
`TelemetryPanel`	4 gauge cards (speed, temp, fuel, battery) with color-coded status bars
`AlertsPanel`	Scrollable alert feed with severity badges and acknowledge functionality
`WorkOrdersPanel`	Workflow management with 5-step progress bar and role-based action buttons

9.2 Real-Time Updates

The dashboard uses a custom `useWebSocket` React hook to maintain a persistent connection with the backend. Incoming messages update the React state, causing immediate UI re-renders without polling.

10. REST API Design (CAP OData Simulation)

The backend exposes 6 route groups simulating SAP CAP's OData-style endpoints:

Table 7: API Endpoint Summary

Route Group	Endpoints	Key Operations
`/api/auth`	3	Login, register, get current user
`/api/devices`	3	List, get by ID, create (Admin-only)
`/api/telemetry`	2	Latest readings, device history
`/api/alerts`	2	List alerts, acknowledge
`/api/workorders`	5	List, approve, assign, start, complete
`/api/simulation`	4	Start, stop, inject fault, get status

11. Technology Stack

Table 8: Technology Stack with SAP BTP Equivalents

Layer	Technology Used	SAP BTP Equivalent
Backend Runtime	Node.js + Express + TypeScript	SAP CAP (Node.js runtime)
Frontend	React + Vite + TypeScript	SAP Fiori / UI5
Database	SQLite via sql.js (WASM)	SAP HANA Cloud
Messaging	Node.js EventEmitter	SAP Event Mesh
Real-time	WebSocket (ws library)	SAP Event Mesh WebSocket
Authentication	JWT + bcrypt	SAP XSUAA
Styling	Vanilla CSS (dark theme)	SAP Fiori Theming

12. Setup and Execution

12.1 Prerequisites

- Node.js v18 or later
- npm v8 or later
- No SAP BTP account required

12.2 Installation

```
# Install backend dependencies
cd iot-fleet-monitor/backend
npm install

# Install frontend dependencies
cd ../frontend
npm install

# Seed the database with demo data
cd ../backend
npm run seed
```

Listing 1: Installation Commands

12.3 Running the Application

```
# Terminal 1: Start backend (port 3001)
cd backend
npm run dev

# Terminal 2: Start frontend (port 5173)
cd frontend
npm run dev
```

Listing 2: Start Commands

12.4 Demo Credentials

Role	Username	Password
Admin	admin	admin123
Operator	operator	operator123
Technician	technician	tech123

13. Conclusion

This project successfully demonstrates how SAP BTP's core services – HANA, Event Mesh, XSUAA, Workflow Management, CAP, and Fiori – can be simulated locally to build a fully functional IoT Fleet Monitoring System. The implementation covers:

- **Event-driven architecture** with decoupled pub/sub channels mirroring SAP Event Mesh
- **CAP service patterns** with domain-specific service classes and a structured data model following the O2C reuse pattern
- **Enterprise workflow** with a 5-step state machine and role-based approvals
- **Real-time monitoring** via WebSocket for live dashboard updates
- **Production-grade security** with JWT-based XSUAA simulation and role-based access control

The system processes telemetry from 5 simulated vehicles in real time, automatically detects anomalies against configurable thresholds, and creates actionable maintenance work orders – demonstrating the complete IoT-to-action pipeline that SAP BTP enables for enterprise customers.